SECTION 1

Runtime server and the @workflow decorator

- **Data plane library:** The runtime server ships in the SDK as the container entrypoint; it is not a separate platform microservice listening beside the developer's code.
- **Decorator metadata:** `@workflow` attaches scaling, resources, `response_mode`, timeouts, and retries to the function via `_workflow_metadata` without changing local call semantics.
- **Intent-first handlers:** The workflow receives kwargs and returns a dict; HTTP parsing, routing, and mode-specific response shaping stay in the runtime.
- **Dual consumers:** The deploy CLI reads metadata for infrastructure; the runtime reads the same dict for timeouts, mode, and in-container retry policy.
- **One workflow per image:** `WORKFLOW_NAME` selects which decorated function a replica serves, preserving independent deploy and scale per workflow.
- **Chapter 2 payoff:** The decorator is the deploy contract promised early: name, `api_path`, scale bounds, and how callers wait.

SECTION 2

Sync, stream, and async modes

- **Sync (default):** The HTTP request blocks until the function returns JSON—right for short orchestration under proxy timeouts, with trace headers bound before the first `platform.*` call.
- **Stream:** `response_mode="stream"` maps yields to server-sent events; token chunks can surface while tool-call deltas stay buffered so clients see text, not half-formed JSON.
- **Async:** The handler returns HTTP 202 with a `job_id` and runs the function in a background task—long research or batch work without holding a browser connection for minutes.
- **Blocking work on the loop:** Sync handlers run the developer function in a thread with `asyncio.wait_for` so slow model waits do not starve the ASGI loop.
- **Mode mismatch hurts:** Chat that needs tokens should not use sync-only responses; multi-minute jobs should not pretend to be a single blocking POST.
- **Same business logic:** Only the HTTP contract at the edge changes; the function still composes platform services.

SECTION 3

Async job lifecycle

- **Control-plane job rows:** `create_job` stores status, serialized input, assigned endpoint, and room for progress, result, error, and checkpoint JSON.
- **Three consumers:** Pollers read status; the runtime writes progress and outcomes; orphan recovery needs the original body and which replica owned the run.
- **Polling API:** Clients call `GET /jobs/{job_id}` on the Workflow Service—not the workflow pod—so job truth survives replica death.
- **Progress without passing ids:** Context variables let `update_job_progress` run inside the function without threading job ids through every helper.
- **Checkpoints:** `save_checkpoint` and `load_checkpoint` let retries resume expensive stages instead of replaying the entire pipeline from scratch.
- **Terminal states:** `complete_job` and `fail_job` make polling deterministic; cancellation is best-effort between stages, not a guarantee mid-provider call.

SECTION 4

Production readiness (concurrency, retries, health)

- **Concurrency:** Uvicorn's event loop multiplexes I/O-bound platform calls; production often runs multiple worker processes per container when workloads are wait-heavy.
- **Layered retries:** Incoming HTTP retries, SDK retries to platform services, and provider retries inside the Model Service address different failure classes—confusing them amplifies quota problems.
- **Idempotency awareness:** Non-idempotent tool or side-effect calls need explicit policy before automatic retries duplicate real-world effects.
- **Liveness vs readiness:** Live means the process should restart; ready means this replica should take traffic—failing live during a dependency outage causes restart storms.
- **Timeouts at the runtime:** `timeout_seconds` on the decorator is enforced where the function runs, not only in Kubernetes—otherwise hung provider calls hold connections forever.
- **AI-specific stress:** Seconds-long LLM waits make thread offloading and probe semantics first-class concerns, not optional polish.

SECTION 5

Workflow composition

- **`platform.workflows.call`:** Parents invoke children through the same gateway paths external clients use, avoiding merged codebases and merged resource envelopes.
- **Trace and identity forwarding:** Parent `trace_id` and caller context must propagate so child spans nest in one waterfall.
- **`call_parallel`:** Independent child workflows run concurrently; wall time tracks the slowest branch plus synthesis, not the sum of serial HTTP calls.
- **Mode-adaptive `call`:** Sync children return JSON; stream children are consumed to completion; async children poll to a terminal job state with a bounded parent timeout.
- **Cycle and depth control:** Registry-aware graphs or limits prevent A calls B calls A runaway.
- **Domain boundaries:** Search, classify, and summarize stay separate deployable units that teams evolve on their own cadence.

SECTION 6

Service contract, deployment pipeline, and runtime management

- **Control plane gRPC:** RegisterWorkflow, Deploy, rollback, and job RPCs split catalog and actuation from the runtime's execution role.
- **`genai-platform deploy`:** Discover decorated functions, build and push an image, register the spec, deploy replicas, and publish the gateway route when readiness passes.
- **Dynamic routing:** The Workflow Service tells the gateway which endpoints serve each `api_path` as rolls add and drain replicas—static ingress files cannot track that alone.
- **Kubernetes mapping:** Decorator scale and resource fields become Deployments, probes on live and ready paths, and HorizontalPodAutoscaler targets.
- **Rolling updates:** New pods must pass readiness before receiving traffic; drain timeouts should cover in-flight sync work up to the workflow timeout.
- **End-to-end request:** Gateway auth and trace start, route to a ready replica, SDK retries on blips, traced platform calls, and a response shaped by mode—without the author hand-writing orchestration YAML.